

BasKu – Bus Ticket Booking

Web Storage – sessionStorage Project

JOB SHEET 13 – WEB STORAGE (sessionStorage)

Student Name: Syedhaziq

Project: BasKu Bus Ticket Booking Wizard (self-made project)

Lecturer: MOHD SHUKRI BIN MUHAMAD HUSIN

Programme: Diploma in Computer Technology (Big Data), TVETMARA Lumut

Files: haziq-13-bus-ticket-booking-session-storage.html, .css, .js

1. Project Description

BasKu is a bus ticket booking wizard with five steps: Trip, Seats, Passengers, Review and Ticket. Every choice is saved in `sessionStorage` as the user goes along. If the page is refreshed, the booking continues from the same step. If the page is opened in a new tab, a new empty booking starts, and when the tab is closed everything is deleted. This is a good fit for `sessionStorage` because a booking form holds personal details (names and phone number) that should not stay in the browser after the tab is closed.

2. Main Features

- Step 1: choose From, To, date, departure time and number of passengers (1–4); the sample fare is shown immediately.
- Step 2: bus seat map with 40 seats; some seats are shown as taken, and exactly as many seats as passengers must be chosen.
- Step 3: passenger names and a Malaysian contact phone number, with validation.
- Step 4: review page showing route, seats, passengers and total price (fare + RM 1.00 service fee per ticket).
- Step 5: ticket with a random booking reference; the ticket can be printed.
- Session Inspector panel shows the `sessionStorage` keys, their sizes, the saved JSON and when this tab's session started.
- "Clear this tab's booking" and "New booking" buttons remove the saved booking.

3. Screenshots

 **Basku Bus Ticket Booking**

Job Sheet 13 · Web Storage – sessionStorage · by Haziq

Sample fares – demo only

1. Trip

2. Seats

3. Passengers

4. Review

5. Ticket

1. Choose your trip

From

Kuala Lumpur

To

Penang

Date

10/11/2026

Departure

08:00

Passengers

2

Fare: RM 37.00 per passenger (+ RM 1.00 service fee) · about 355 km

← Back

Next →

Session Inspector

This tab's session started at 3:42:28 PM.

Key	Stores	Size
haziq13_bus_draft	Booking in progress	181
haziq13_bus_started	When this tab's session began	45

► View saved JSON

Try it: fill in a few steps, then refresh – your booking stays. Open this page in a *new tab* – it starts fresh, because `sessionStorage` belongs to one tab only. Close the tab and everything is deleted.

Clear this tab's booking

Figure 1 – Step 1: choose the trip (the form is saved in sessionStorage as it changes).



1. Trip

2. Seats

3. Passengers

4. Review

5. Ticket

2. Pick your seats

Kuala Lumpur → Penang · Sun, 11 Oct 2026 · 08:00

☐ Free ☒ Yours ☐ Taken

Driver

1	1A	1B	1C	1D
2	2A	2B	2C	2D
3	3A	3B	3C	3D
4	4A	4B	4C	4D
5	5A	5B	5C	5D
6	6A	6B	6C	6D
7	7A	7B	7C	7D
8	8A	8B	8C	8D
9	9A	9B	9C	9D
10	10A	10B	10C	10D

Selected 2 of 2: 1A, 1B

← Back

Next →

Session Inspector

This tab's session started at 3:42:28 PM.

Key	Stores	Size
haziq13_bus_draft	Booking in progress	226
haziq13_bus_started	When this tab's session began	45

► View saved JSON

Try it: fill in a few steps, then refresh – your booking stays. Open this page in a *new tab* – it starts fresh, because `sessionStorage` belongs to one tab only. Close the tab and everything is deleted.

Clear this tab's booking

Figure 2 – Step 2: seat map with free, selected and taken seats.

BasKu Bus Ticket Booking

Job Sheet 13 · Web Storage – sessionStorage · by Haziq

Sample fares – demo only

1. Trip

2. Seats

3. Passengers

4. Review

5. Ticket

3. Passenger details

Passenger 1 · Seat 1A

Full name

Syed Haziq

Passenger 2 · Seat 1B

Full name

Aina Sofea

Contact phone (Malaysia)

012-3456789

Nothing here is kept after you close this tab.

Enter a valid Malaysian mobile number, e.g. 012-3456789.

← Back

Next →

Session Inspector

This tab's session started at 3:42:28 PM.

Key	Stores	Size
haziq13_bus_draft	Booking in progress	262
haziq13_bus_started	When this tab's session began	45

► View saved JSON

Try it:

fill in a few steps, then refresh – your booking stays. Open this page in a new tab – it starts fresh, because sessionStorage belongs to one tab only. Close the tab and everything is deleted.

Clear this tab's booking

Figure 3 – Step 3: passenger details.

 **BasKu Bus Ticket Booking**

Job Sheet 13 · Web Storage – sessionStorage · by Haziq

Sample fares – demo only

1. Trip

2. Seats

3. Passengers

4. Review

5. Ticket

5. Your ticket

Booking reference

BUS-VDDFQ6

Kuala Lumpur → Penang

Sun, 11 Oct 2026, 08:00

Seats: **1A, 1B**

1. Syed Haziq (1A)

2. Aina Sofea (1B)

Phone: 012-3456789

Total paid: **RM 76.00**

Booked 10/10/2026, 3:42:33 PM

Print ticket

New booking

Session Inspector

This tab's session started at 3:42:28 PM.

Key	Stores	Size
haziq13_bus_draft	Booking in progress	483
haziq13_bus_started	When this tab's session began	45

► View saved JSON

Try it: fill in a few steps, then refresh – your booking stays. Open this page in a *new tab* – it starts fresh, because `sessionStorage` belongs to one tab only. Close the tab and everything is deleted.

Clear this tab's booking

Booking confirmed: BUS-VDDFQ6

Figure 4 – Step 5: the confirmed ticket and the Session Inspector.

4. How sessionStorage Is Used

All keys start with the prefix `haziq13_bus_`.

Key	Stores	Example value
<code>haziq13_bus_draft</code>	Booking in progress (step, trip, seats, passengers, phone, ticket)	<code>{"step":2,"trip":{"from":"Kuala Lumpur","to":"Penang","date":"2026-10-11","time":"08:00","pax":2},"seats":["1A","1B"],...}</code>
<code>haziq13_bus_started</code>	When this tab's session began	<code>"2026-10-10T07:42:28.000Z"</code>

Web Storage methods and techniques used

Technique	Where it is used
<code>sessionStorage.setItem()</code>	Saving the booking after every change and the session start time
<code>sessionStorage.getItem()</code>	Restoring the booking when the page loads or is refreshed
<code>sessionStorage.removeItem()</code>	New booking and "Clear this tab's booking"
<code>sessionStorage.length</code> and <code>key(i)</code>	Session Inspector lists the keys used by the app
<code>JSON.stringify()</code> / <code>JSON.parse()</code>	Saving the whole booking object as one JSON string
Data validation on load	Corrupted or old data falls back to a fresh booking
<code>try / catch</code>	Handles blocked or full storage

sessionStorage compared with localStorage

	sessionStorage	localStorage
Lifetime	Until the tab is closed	Until it is deleted
Scope	One tab only (a new tab is empty)	All tabs of the same site
Survives refresh	Yes	Yes
Used in this project for	A booking in progress with personal details	Not used (see Job Sheet 12)

5. Testing

The app was tested in a real browser (Chromium). Results:

- Choosing the same city for From and To was blocked with an error message.
- Pressing Next with no seats selected was blocked; choosing a third seat for two passengers showed "You can only choose 2 seats."
- Seats 1A and 1B were selected, the page was refreshed: both seats were still selected on the same step.
- An invalid phone number (0123) was rejected; 012-3456789 was accepted.
- Refreshing on the review page kept the booking; total for 2 passengers on Kuala Lumpur → Penang was RM 76.00.

- After confirming, the ticket and its reference stayed the same after a refresh.
- A new tab opened on Step 1 with an empty booking (sessionStorage is separate per tab).
- "New booking" removed the saved booking; no JavaScript errors were reported in the console.

6. How to Run

Open `haziq-13-bus-ticket-booking-session-storage.html` in a browser. No server or Internet connection is needed. The folder `split` contains the same app as separate HTML, CSS and JavaScript files (keep the three files together in one folder).

7. Source Code – haziq-13-bus-ticket-booking-session-storage.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>BasKu – Bus Ticket Booking (sessionStorage)</title>
<link rel="stylesheet" href="haziq-13-bus-ticket-booking-session-storage.css">
</head>
<body>
<header class="top">
<div>
<h1>&img alt="BasKu logo" data-bbox="125 231 145 241"/> BasKu Bus Ticket Booking</h1>
<p>Job Sheet 13 · Web Storage – sessionStorage · by Haziq</p>
</div>
<span class="badge">Sample fares – demo only</span>
</header>

<main class="layout">
<section class="card main">
<ol id="progress" class="progress" aria-label="Booking steps"></ol>
<div id="stepView"></div>
<p id="msg" class="msg" role="alert"></p>
<div id="navBar" class="nav"></div>
</section>

<aside class="card side">
<h2>Session Inspector</h2>
<p id="sessionInfo" class="hint"></p>
<div class="scroll">
<table id="storageTable">
<thead><tr><th>Key</th><th>Stores</th><th>Size</th></tr></thead>
<tbody></tbody>
</table>
</div>
<details>
<summary>View saved JSON</summary>
<pre id="rawView"></pre>
</details>
<div class="tip">
<strong>Try it:</strong> fill in a few steps, then refresh – your booking stays.
Open this page in a <em>new tab</em> – it starts fresh, because <code>sessionStorage</code> belongs to one tab
only.
Close the tab and everything is deleted.
</div>
<button id="clearBtn" class="danger" type="button">Clear this tab's booking</button>
</aside>
</main>

<p id="toast" class="toast" role="status" aria-live="polite"></p>
<script src="haziq-13-bus-ticket-booking-session-storage.js"></script>
</body>
</html>
```

8. Source Code – haziq-13-bus-ticket-booking-session-storage.css

```
:root{--bg:#eef6f5;--card:#fff;--text:#16302e;--muted:#5c7774;--line:#d3e3e1;--brand:#0f766e;--brand-d:#0b5a54;--
chip:#e6f4f2;--danger:#b91c1c;--taken:#c9d3d2}
*{box-sizing:border-box}
body{margin:0;font-family:"Segoe UI",Arial,sans-serif;background:var(--bg);color:var(--text);line-height:1.45}
.top{display:flex;justify-content:space-between;align-items:center;gap:12px;flex-wrap:wrap;padding:16px
24px;background:var(--brand);color:#fff}
.top h1{margin:0;font-size:1.45rem}
.top p{margin:2px 0 0;font-size:.85rem;opacity:.9}
.badge{background:rgba(255,255,255,.18);padding:5px 12px;border-radius:999px;font-size:.8rem}
.layout{max-width:1100px;margin:0 auto;padding:20px;display:grid;grid-template-columns:1.7fr 1fr;gap:18px;align-
items:start}
.card{background:var(--card);border:1px solid var(--line);border-radius:14px;padding:18px}
h2{margin:0 0 12px;font-size:1.15rem}
.progress{list-style:none;display:flex;gap:6px;margin:0 0 18px;padding:0}
.progress li{flex:1;text-align:center;font-size:.78rem;color:var(--muted);padding-bottom:8px;border-bottom:4px solid
var(--line)}
```

```

.progress li.done{color:var(--brand);border-color:var(--brand)}
.progress li.active{color:var(--brand-d);font-weight:700;border-color:var(--brand-d)}
.grid2{display:grid;grid-template-columns:1fr 1fr;gap:12px}
label{display:flex;flex-direction:column;gap:4px;font-size:.85rem;color:var(--muted)}
input,select{padding:9px 10px;border:1px solid var(--line);border-radius:8px;font:inherit;color:var(--text);background:#fff}
input:focus,select:focus{outline:2px solid var(--brand);outline-offset:1px}
.fare{margin:14px 0 0;padding:10px 12px;background:var(--chip);border-radius:10px}
.msg{min-height:1.4em;margin:12px 0 0;color:var(--danger);font-weight:600}
.nav{display:flex;justify-content:space-between;gap:10px;margin-top:10px}
button{font:inherit;cursor:pointer}
.primary,.ghost,.danger{padding:10px 18px;border-radius:10px;border:1px solid transparent;font-weight:600}
.primary{background:var(--brand);color:#fff}
.primary:hover{background:var(--brand-d)}
.ghost{background:#fff;color:var(--text);border-color:var(--line)}
.danger{background:#fff;color:var(--danger);border-color:var(--danger);width:100%;margin-top:12px}
.bus{display:inline-block;border:2px solid var(--line);border-radius:18px;padding:14px 16px;background:#fafdfc}
.bus .front{text-align:center;color:var(--muted);font-size:.8rem;margin-bottom:8px}
.seatrow{display:flex;align-items:center;gap:6px;margin-bottom:6px}
.seatrow .no{width:20px;font-size:.75rem;color:var(--muted);text-align:right}
.aisle{width:24px}
.seat{width:42px;height:38px;border-radius:8px 8px 4px 4px;border:1px solid var(--brand);background:#fff;color:var(--brand-d);font-size:.78rem;font-weight:600}
.seat:hover:not(:disabled){background:var(--chip)}
.seat.sel{background:var(--brand);color:#fff}
.seat.disabled{background:var(--taken);border-color:var(--taken);color:#7d8c8b;cursor:not-allowed}
.legend{display:flex;gap:14px;font-size:.8rem;color:var(--muted);margin:10px 0}
.legend i{display:inline-block;width:14px;height:14px;border-radius:4px;vertical-align:-2px;margin-right:4px;border:1px solid var(--brand)}
.pcard{border:1px solid var(--line);border-radius:12px;padding:12px;margin-bottom:12px;background:#fafdfc}
.pcard h3{margin:0 0 8px;font-size:.95rem}
table{border-collapse:collapse;width:100%;font-size:.88rem}
th,td{border:1px solid var(--line);padding:7px 9px;text-align:left;vertical-align:top}
th{background:var(--chip)}
.sum th{width:34%}
.ticket{border:2px dashed var(--brand);border-radius:14px;padding:18px;background:#fff}
.ticket .ref{font-size:1.6rem;font-weight:800;letter-spacing:2px;color:var(--brand-d)}
.ticket .route{font-size:1.3rem;font-weight:700;margin:6px 0}
.hint{color:var(--muted);font-size:.82rem;margin:0 0 10px}
.scroll{overflow-x:auto}
code{background:var(--chip);padding:1px 5px;border-radius:5px;font-size:.85em}
details{margin:10px 0}
pre{background:#10211f;color:#bff3ea;padding:10px;border-radius:8px;overflow:auto;max-height:220px;font-size:.72rem}
.tip{background:var(--chip);border-radius:10px;padding:10px 12px;font-size:.82rem;margin-top:8px}
.toast{position:fixed;left:50%;bottom:22px;transform:translateX(-50%) translateY(20px);background:#16302e;color:#fff;padding:10px 18px;border-radius:999px;opacity:0;pointer-events:none;transition:.25s}
.toast.show{opacity:1;transform:translateX(-50%) translateY(0)}
@media (max-width:820px){.layout{grid-template-columns:1fr;padding:12px}.grid2{grid-template-columns:1fr}}
@media print{body *{visibility:hidden}#ticketBox,#ticketBox *{visibility:visible}#ticketBox{position:absolute;left:0;top:0;width:100%}}

```

9. Source Code – haziq-13-bus-ticket-booking-session-storage.js

```
"use strict";

/* ----- Config ----- */
const PREFIX = "haziq13_bus_";
const KEYS = { draft: PREFIX + "draft", started: PREFIX + "started" };
const KEY_INFO = {
  [KEYS.draft]: "Booking in progress",
  [KEYS.started]: "When this tab's session began"
};
const CITIES = ["Ipoh", "Johor Bahru", "Kota Bharu", "Kuala Lumpur", "Kuantan", "Melaka", "Penang"];
const DIST = [ // approximate road distance (km), used only to make sample fares
  ["Kuala Lumpur", "Ipoh", 205], ["Kuala Lumpur", "Penang", 355], ["Kuala Lumpur", "Johor Bahru", 330],
  ["Kuala Lumpur", "Kuantan", 260], ["Kuala Lumpur", "Kota Bharu", 470], ["Kuala Lumpur", "Melaka", 150],
  ["Ipoh", "Penang", 160], ["Ipoh", "Johor Bahru", 535], ["Ipoh", "Kuantan", 330],
  ["Ipoh", "Kota Bharu", 410], ["Ipoh", "Melaka", 350], ["Penang", "Johor Bahru", 690],
  ["Penang", "Kuantan", 520], ["Penang", "Kota Bharu", 360], ["Penang", "Melaka", 500],
  ["Johor Bahru", "Kuantan", 380], ["Johor Bahru", "Kota Bharu", 700], ["Johor Bahru", "Melaka", 200],
  ["Kuantan", "Kota Bharu", 360], ["Kuantan", "Melaka", 370], ["Kota Bharu", "Melaka", 620]
];
const KM = {};
DIST.forEach(([a, b, km]) => { KM[pairKey(a, b)] = km; });
const TIMES = ["08:00", "10:30", "14:00", "20:30", "23:30"];
const ROWS = 10, COLS = ["A", "B", "C", "D"];
const STEPS = ["Trip", "Seats", "Passengers", "Review", "Ticket"];
const SERVICE_FEE = 100; // sen per ticket

function pairKey(a, b) { return [a, b].sort().join("|"); }

/* ----- sessionStorage helpers (wrapped in try/catch) ----- */
const store = {
  get(key, fallback) {
    try {
      const raw = sessionStorage.getItem(key);
      return raw === null ? fallback : JSON.parse(raw);
    } catch (e) { return fallback; }
  },
  set(key, value) {
    try { sessionStorage.setItem(key, JSON.stringify(value)); return true; }
    catch (e) { toast("Could not save: storage is full or blocked."); return false; }
  },
  remove(key) { try { sessionStorage.removeItem(key); } catch (e) { /* ignore */ } },
  keys() {
    const out = [];
    try {
      for (let i = 0; i < sessionStorage.length; i++) {
        const k = sessionStorage.key(i);
        if (k && k.startsWith(PREFIX)) out.push(k);
      }
    } catch (e) { /* ignore */ }
    return out.sort();
  }
};

/* ----- Utilities ----- */
const $ = id => document.getElementById(id);
const esc = s => String(s).replace(/[\<>'"]/, g => ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", "'": "&quot;", '"': "&#39;" }[g]));
const money = sen => "RM " + (sen / 100).toFixed(2);
const pad = n => String(n).padStart(2, "0");
const isoDate = d => d.getFullYear() + "-" + pad(d.getMonth() + 1) + "-" + pad(d.getDate());
function addDays(n) { const d = new Date(); d.setDate(d.getDate() + n); return isoDate(d); }
let toastTimer;
function toast(msg) {
  const t = $("toast");
  t.textContent = msg; t.classList.add("show");
  clearTimeout(toastTimer);
  toastTimer = setTimeout(() => t.classList.remove("show"), 2200);
}

/* ----- State ----- */
function freshState() {
  return {
    step: 1,
```

```

    trip: { from: "Kuala Lumpur", to: "Penang", date: addDays(1), time: "08:00", pax: 1 },
    seats: [], seatSig: "", passengers: [], phone: "", booking: null
  };
}
function loadState() {
  const d = store.get(KEYS.draft, null), f = freshState();
  if (!d || typeof d !== "object") return f;
  try {
    const t = d.trip || {};
    const s = {
      step: Number.isInteger(d.step) && d.step >= 1 && d.step <= 5 ? d.step : 1,
      trip: {
        from: CITIES.includes(t.from) ? t.from : f.trip.from,
        to: CITIES.includes(t.to) ? t.to : f.trip.to,
        date: /^\\d{4}-\\d{2}-\\d{2}$/.test(t.date) ? t.date : f.trip.date,
        time: TIMES.includes(t.time) ? t.time : f.trip.time,
        pax: [1, 2, 3, 4].includes(t.pax) ? t.pax : 1
      },
      seats: Array.isArray(d.seats) ? d.seats.filter(x => typeof x === "string") : [],
      seatSig: typeof d.seatSig === "string" ? d.seatSig : "",
      passengers: Array.isArray(d.passengers) ? d.passengers.map(String) : [],
      phone: typeof d.phone === "string" ? d.phone : "",
      booking: d.booking && typeof d.booking === "object" ? d.booking : null
    };
    if (s.step === 5 && !s.booking) s.step = 4;
    return s;
  } catch (e) { return f; }
}
let state = loadState();
if (store.get(KEYS.started, null) === null) store.set(KEYS.started, new Date().toISOString());
function save() { store.set(KEYS.draft, state); renderInspector(); }

/* ----- Trip helpers ----- */
const tripSig = () => [state.trip.from, state.trip.to, state.trip.date, state.trip.time].join("|");
function fareSen() {
  const km = KM[pairKey(state.trip.from, state.trip.to)] || 0;
  return Math.round(Math.round((5 + km * 0.09) * 2) / 2 * 100);
}
function hash(str) { let h = 5381; for (const c of str) h = ((h << 5) + h + c.charCodeAt(0)) >>> 0; return h; }
const isTaken = seat => hash(tripSig() + seat) % 100 < 28;
const seatOrder = id => parseInt(id, 10) * 10 + COLS.indexOf(id.slice(-1));
const sortSeats = list => list.slice().sort((a, b) => seatOrder(a) - seatOrder(b));
const totalSen = () => (fareSen() + SERVICE_FEE) * state.trip.pax;
const fmtDate = iso => new Date(iso + "T00:00:00").toLocaleDateString("en-GB", { weekday: "short", day: "numeric", month: "short", year: "numeric" });

/* ----- Rendering ----- */
function renderProgress() {
  $("progress").innerHTML = STEPS.map((s, i) => {
    const n = i + 1, cls = n < state.step ? "done" : n === state.step ? "active" : "";
    return `<li class="${cls}">${n}. ${s}</li>`;
  }).join("");
}
function opts(list, sel) { return list.map(x => `<option${x === sel ? " selected" : ""}>${esc(x)}</option>`).join(""); }
function fareLine() {
  const km = KM[pairKey(state.trip.from, state.trip.to)];
  return state.trip.from === state.trip.to ? "Choose two different cities."
    : `Fare: <strong>${money(fareSen())}</strong> per passenger (+ ${money(SERVICE_FEE)} service fee) · about ${km} km`;
}
function renderStep() {
  const t = state.trip, v = $("stepView");
  if (state.step === 1) {
    v.innerHTML = `<h2>1. Choose your trip</h2><div class="grid2">
      <label>From<select data-f="from">${opts(CITIES, t.from)}</select></label>
      <label>To<select data-f="to">${opts(CITIES, t.to)}</select></label>
      <label>Date<input type="date" data-f="date" min="${addDays(0)}" value="${esc(t.date)}"></label>
      <label>Departure<select data-f="time">${opts(TIMES, t.time)}</select></label>
      <label>Passengers<select data-f="pax">${opts(["1", "2", "3", "4"], String(t.pax))}</select></label></div>
      <p class="fare" id="fareLine">${fareLine()}</p>`;
  } else if (state.step === 2) {
    let rows = "";
    for (let r = 1; r <= ROWS; r++) {
      const btn = c => { const id = r + c, sel = state.seats.includes(id);
        return `<button type="button" class="seat${sel ? " sel" : ""}" data-seat="${id}"${isTaken(id) ? " disabled" : ""}>${id}</button>`;
      };
      rows += `<div class="seatrow"><span class="no">${r}</span>${btn("A")}${btn("B")}<span class="aisle">
        </span>${btn("C")}${btn("D")}</div>`;
    }
  }
}

```

```

    }
    v.innerHTML = `<h2>2. Pick your seat${t.pax > 1 ? "s" : ""}</h2>
    <p class="hint">${esc(t.from)} → ${esc(t.to)} · ${fmtDate(t.date)} · ${t.time}</p>
    <div class="legend"><span><i style="background:#fff"></i>Free</span><span><i style="background:var(--brand)">
</i>Yours</span><span><i style="background:var(--taken);border-color:var(--taken)"></i>Taken</span></div>
    <div class="bus"><div class="front">👤 Driver</div>${rows}</div>
    <p class="fare" id="seatCount"></p>`;
    renderSeatCount();
  } else if (state.step === 3) {
    const seats = sortSeats(state.seats);
    let cards = "";
    for (let i = 0; i < t.pax; i++) {
      cards += `<div class="pcard"><h3>Passenger ${i + 1} · Seat ${esc(seats[i] || "?")}</h3>
      <label>Full name<input type="text" data-name="${i}" maxlength="60" value="${esc(state.passengers[i] || "")}"
autocomplete="off"></label></div>`;
    }
    v.innerHTML = `<h2>3. Passenger details</h2>${cards}
    <label>Contact phone (Malaysia)<input type="tel" data-f="phone" placeholder="012-3456789"
value="${esc(state.phone)}"></label>
    <p class="hint">Nothing here is kept after you close this tab.</p>`;
  } else if (state.step === 4) {
    const seats = sortSeats(state.seats);
    v.innerHTML = `<h2>4. Review your booking</h2>
    <table class="sum">
    <tr><th>Route</th><td>${esc(t.from)} → ${esc(t.to)}</td></tr>
    <tr><th>Departure</th><td>${fmtDate(t.date)}, ${t.time}</td></tr>
    <tr><th>Seats</th><td>${seats.map(esc).join(", ")}</td></tr>
    <tr><th>Passengers</th><td>${state.passengers.slice(0, t.pax).map((n, i) => `${i + 1}. ${esc(n)}
(${esc(seats[i])})`).join("<br>")}</td></tr>
    <tr><th>Phone</th><td>${esc(state.phone)}</td></tr>
    <tr><th>Fare</th><td>${t.pax} × ${money(fareSen())} + ${t.pax} × ${money(SERVICE_FEE)} service fee</td></tr>
    <tr><th>Total</th><td><strong>${money(totalSen())}</strong></td></tr></table>`;
  } else {
    const b = state.booking;
    v.innerHTML = `<h2>5. Your ticket</h2><div class="ticket" id="ticketBox">
    <small>Booking reference</small><div class="ref">${esc(b.ref)}</div>
    <div class="route">${esc(b.from)} → ${esc(b.to)}</div>
    <div>${fmtDate(b.date)}, ${esc(b.time)}</div>
    <p>Seats: <strong>${b.seats.map(esc).join(", ")}</strong></p>
    <p>${b.passengers.map((n, i) => `${i + 1}. ${esc(n)} (${esc(b.seats[i])})`).join("<br>")}</p>
    <p>Phone: ${esc(b.phone)}<br>Total paid: <strong>${money(b.total)}</strong></p>
    <small>Booked ${new Date(b.bookedAt).toLocaleString()}</small></div>`;
  }
  renderNav();
}
function renderSeatCount() {
  const el = $("seatCount");
  if (el) el.innerHTML = `Selected <strong>${state.seats.length}</strong> of ${state.trip.pax}:
${sortSeats(state.seats).join(", ") || "none yet"}`;
}
function renderNav() {
  const s = state.step;
  $("navBar").innerHTML = s === 5
    ? `<button class="ghost" type="button" data-act="print">Print ticket</button><button class="primary"
type="button" data-act="new">New booking</button>`
    : `<button class="ghost" type="button" data-act="back">${s === 1 ? "disabled" : ""}>← Back</button>
    <button class="primary" type="button" data-act="next">${s === 4 ? "Confirm booking" : "Next →"}</button>`;
}
function renderInspector() {
  const rows = store.keys().map(k => {
    let size = 0;
    try { size = k.length + (sessionStorage.getItem(k) || "").length; } catch (e) { /* ignore */ }
    return `<tr><td><code>${esc(k)}</code></td><td>${esc(KEY_INFO[k] || "Other")}</td><td>${size}</td></tr>`;
  });
  $("storageTable").querySelector("tbody").innerHTML = rows.length ? rows.join("") : `<tr><td colspan="3">Nothing
stored in this tab yet.</td></tr>`;
  const started = store.get(KEYS.started, null);
  $("sessionInfo").textContent = started ? `This tab's session started at " + new Date(started).toLocaleTimeString()
+ "." : "`;
  let raw = "";
  try { raw = JSON.stringify(JSON.parse(sessionStorage.getItem(KEYS.draft) || "null"), null, 1) || "(empty)"; }
  catch (e) { raw = "(unreadable)"; }
  $("rawView").textContent = raw;
}
/* ----- Validation and navigation ----- */
function validate(step) {
  const t = state.trip;
  if (step === 1) {

```

```

    if (t.from === t.to) return "Departure and destination must be different.";
    if (!t.date || t.date < addDays(0)) return "Choose today or a later date.";
  }
  if (step === 2 && state.seats.length !== t.pax) return `Please select exactly ${t.pax} seat${t.pax > 1 ? "s" : ""}.`;
  if (step === 3) {
    for (let i = 0; i < t.pax; i++) {
      const n = (state.passengers[i] || "").trim();
      if (n.length < 2) return `Enter the full name for passenger ${i + 1}.`;
    }
    const p = state.phone.replace(/[s-]/g, "");
    if (!/^(\\+?60|0)1\\d{8,9}$/.test(p)) return "Enter a valid Malaysian mobile number, e.g. 012-3456789.";
  }
  return "";
}
function goto(step) { state.step = step; $("msg").textContent = ""; save(); renderProgress(); renderStep(); }
function next() {
  const err = validate(state.step);
  if (err) { $("msg").textContent = err; return; }
  if (state.step === 1 && state.seatSig !== tripSig()) { state.seats = []; state.seatSig = tripSig(); }
  if (state.step === 2) {
    state.seats = sortSeats(state.seats);
    state.passengers = Array.from({ length: state.trip.pax }, (_, i) => state.passengers[i] || "");
  }
  if (state.step === 3) state.passengers = state.passengers.map(n => n.trim()).slice(0, state.trip.pax);
  if (state.step === 4) { confirmBooking(); return; }
  goto(state.step + 1);
}
function confirmBooking() {
  const chars = "ABCDEFGHJKLMNPQRSTUVWXYZ23456789";
  let ref = "BUS-";
  for (let i = 0; i < 6; i++) ref += chars[Math.floor(Math.random() * chars.length)];
  const t = state.trip;
  state.booking = { ref, from: t.from, to: t.to, date: t.date, time: t.time, seats: sortSeats(state.seats),
    passengers: state.passengers.slice(0, t.pax), phone: state.phone.trim(), total: totalSen(), bookedAt: new
Date().toISOString() };
  goto(5);
  toast("Booking confirmed: " + ref);
}
function resetBooking() {
  store.remove(KEYS.draft);
  state = freshState();
  $("msg").textContent = "";
  renderProgress(); renderStep(); renderInspector();
}
}

/* ----- Events ----- */
$("stepView").addEventListener("input", e => {
  const el = e.target;
  if (el.dataset.name !== undefined) { state.passengers[Number(el.dataset.name)] = el.value; save(); }
  else if (el.dataset.f === "phone") { state.phone = el.value; save(); }
});
$("stepView").addEventListener("change", e => {
  const f = e.target.dataset.f;
  if (!f || f === "phone") return;
  state.trip[f] = f === "pax" ? Number(e.target.value) : e.target.value;
  if (f === "pax" && state.seats.length > state.trip.pax) state.seats = state.seats.slice(0, state.trip.pax);
  $("msg").textContent = "";
  save();
  $("fareLine").innerHTML = fareLine();
});
$("stepView").addEventListener("click", e => {
  const b = e.target.closest("button[data-seat]");
  if (!b || b.disabled) return;
  const id = b.dataset.seat, i = state.seats.indexOf(id);
  if (i >= 0) state.seats.splice(i, 1);
  else if (state.seats.length >= state.trip.pax) { toast(`You can only choose ${state.trip.pax} seat${state.trip.pax > 1 ? "s" : ""}.`); return; }
  else state.seats.push(id);
  state.seatSig = tripSig();
  b.classList.toggle("sel");
  $("msg").textContent = "";
  save(); renderSeatCount();
});
$("navBar").addEventListener("click", e => {
  const b = e.target.closest("button[data-act]");
  if (!b) return;
  const a = b.dataset.act;
  if (a === "next") next();
});

```

```
    else if (a === "back" && state.step > 1) goto(state.step - 1);
    else if (a === "print") window.print();
    else if (a === "new") resetBooking();
  });
  $("#clearBtn").addEventListener("click", () => {
    if (!confirm("Clear this tab's booking and start again?")) return;
    resetBooking();
    toast("Session booking cleared.");
  });

  /* ----- Start ----- */
  renderProgress();
  renderStep();
  renderInspector();
```